



Android: Architettura Applicazioni, Lifecycle, Sicurezza e Permessi

Sistemi Operativi

Davide Loconte

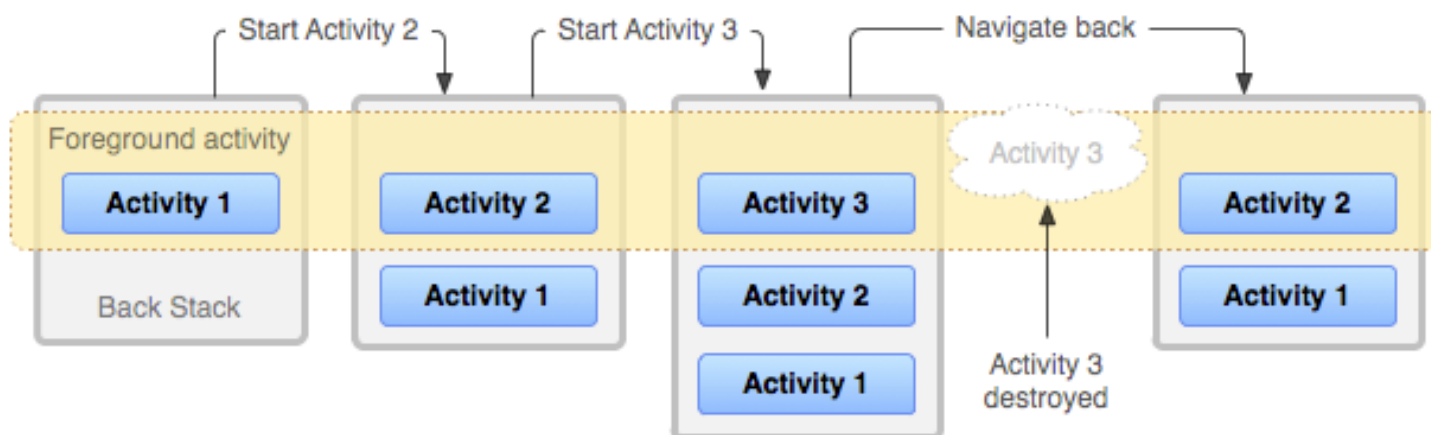
Ingegneria Informatica e dell'Automazione
(A.A. 2024/2025)

- Introduzione
- Componenti delle applicazioni
- Intent
- Android Manifest
- Sicurezza e permessi
- Data storage

Un'applicazione consiste in uno o più componenti

- Ciascun componente costituisce un entry point per il sistema, per un utente o per una altra applicazione
 - Activity
 - Service
 - Broadcast Receiver
 - Content Provider
 - Fragments
 - Intent
- SDK vs NDK -> Nelle slide ci si riferisce al **Software Development Kit**
 - Le applicazioni sviluppate con l'**SDK** in Java o Kotlin Android girano su macchina virtuale ART e sono indipendenti dall'hardware
 - **Native Development Kit (NDK)** consente di sviluppare software nativo per scenari specifici usando i linguaggi C/C++ -> Il codice va ricompilato per ciascuna architettura che si intende supportare

- Una **Activity** rappresenta una singola schermata di una applicazione interattiva (user entry point) tramite cui possono essere realizzate differenti azioni
 - Ogni Activity è **indipendente**
 - Tipicamente c'è una **Activity principale** da cui l'applicazione viene avviata
 - Una applicazione può avere **diverse Activity**
- **Activity Stack** -> Il sistema tiene conto delle schermate precedenti e del loro ordine



Lifecycle Activity 1/2

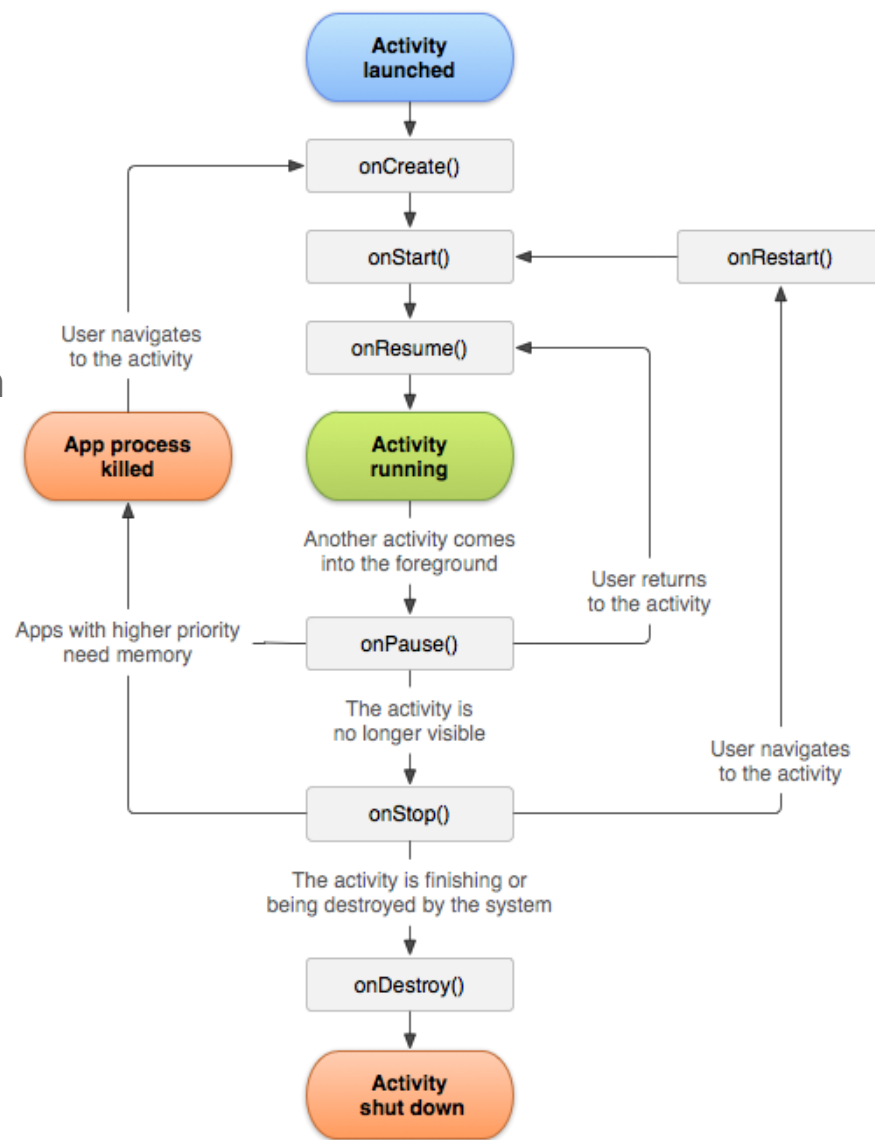


L'activity può essere:

- **attiva o in esecuzione** -> activity in primo piano
- **in pausa** -> oscurata parzialmente (ad esempio se viene sovrapposta da un dialog)
- **terminata** -> quando viene chiusa e una nuova activity passa in primo piano

Visibile lifetime -> da onStart() a onStop()

Foreground lifetime -> da onResume() a onPause()



Lifecycle Activity 2/2



- **onCreate()** -> invocato quando l'Activity è **creata** per la prima volta, è il metodo in cui dovrebbe essere inserito tutto il codice per la fase di **setup dell'Activity**
- **onStart()** -> invocato prima che l'Activity sia **resa visibile** all'utente
- **onResume()** -> chiamato quando l'activity è totalmente visibile e poco prima che l'Activity cominci **l'interazione con l'utente**
- **onPause()** -> invocato quando il sistema sta per ripristinare un'altra Activity
- **onStop()** -> invocato quando l'Activity **non è più visibile** all'utente
- **onDestroy()** -> eseguito alla **chiusura** e **deallocazione** dell'activity

Stati killabili -> onPause(), onStop(), onDestroy()

- L'applicazione può essere chiusa per **mancanza di memoria**

Fragments

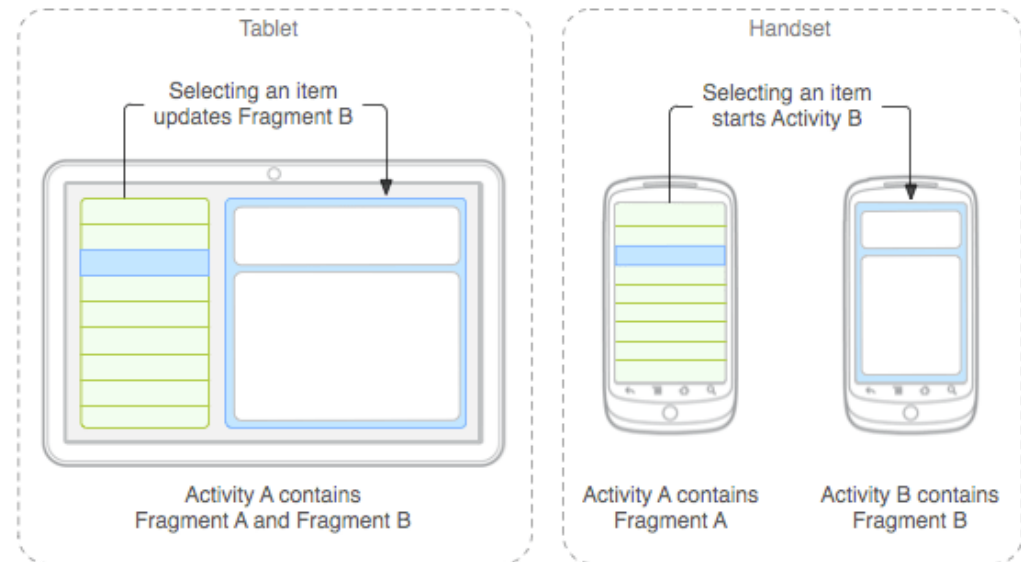


Rappresenta una **componente modulare** dell'interfaccia di un'app Android

- Favorisce la riutilizzabilità dell'interfaccia utente e delle sue componenti
- Consente di implementare layout multi-panel adattabili a diversi dispositivi minimizzando la duplicazione del codice

LifeCycle

- **Ereditati da Activity:** onCreate, onStart, onResume, onPause, onStop, onDestroy
- **Specifici per Fragment:**
 - **onAttach:** il Fragment è stato associato alla sua Activity
 - **onCreateView:** Chiamato tra onCreate e onActivityCreated
 - **onActivityCreated:** Si verifica dopo la creazione delle view dell'Activity

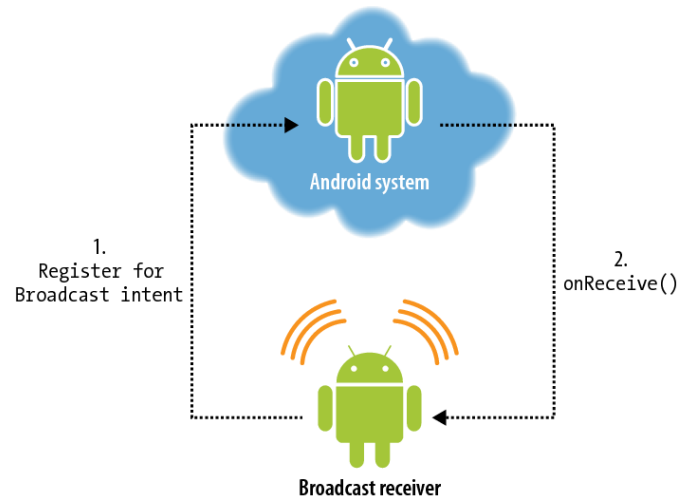


[This Photo](#) by is licensed under [CC BY-SA-NC](#)

- Un **Service** è una componente senza interfaccia visiva che viene eseguita in background
 - Esempio: un Service potrebbe riprodurre musica in background mentre l'utente sta utilizzando un'applicazione diversa o potrebbe recuperare dati in rete senza bloccare l'interazione con un Activity
- Così come un Activity, un **Service** ha dei metodi per la gestione del ciclo di vita che è possibile implementare per monitorare i suoi cambiamenti di stato
 - **onCreate()**
 - **onDestory()**
 - **onStartCommand()** → chiamata quando una data operazione deve essere eseguita in background
 - **onBind()** → chiamata quando viene stabilita una connessione tra l'applicazione richiedente ed il servizio richiesto

Il **Broadcast Receiver** è una componente passiva dell'applicazione che resta in ascolto per degli eventi

- Gli eventi possono essere **generati dal sistema o da un'altra applicazione**
- Affinché un Broadcast Receiver venga chiamato **non è necessario che la relativa applicazione sia in esecuzione**
- Ha un singolo metodo di callback: **onReceive()**



Content Provider 1/2



- Un **Content Provider** è una componente che gestisce l'accesso ai dati di una applicazione
 - Si può usare per **esporre i dati** dell'applicazione anche **all'esterno**
- L'accesso ai dati avviene come se si trattasse di un database
 - I dati del sono restituiti come tabelle di un Database Relazionale
 - Si accede alle singole istanze tramite un **cursore**
- I metodi da implementare sono:
 - **onCreate()** -> inizializza il ContentProvider
 - **update()** -> aggiorna una istanza
 - **remove()** -> rimuove una istanza
 - **insert()** -> inserisce una nuova istanza
 - **query()** -> restituisce il risultato di una query passata come argomento

I **Content Provider** sono identificati da un **URI**:

`content://com.example.transportationprovider/trains/122`



The diagram illustrates the structure of the URI `content://com.example.transportationprovider/trains/122`. It is divided into four parts labeled A, B, C, and D. Part A is `content`, part B is `//com.example.transportationprovider`, part C is `/trains`, and part D is `/122`.

- A -> protocollo (content)
- B -> package del content provider
- C -> tabella
- D -> istanza

ContentResolver -> oggetto comune a tutte le Applicazioni Android che consente di accedere ai Content Provider specificandone l'URI

- Tramite questo oggetto è possibile richiamare i metodi `update()`, `remove()`, `insert()` e `query()` per agire sui dati

Intent 1/3



L'**Intent** è un oggetto generico che rappresenta una operazione tra più **componenti** di Android

Esempi:

- Lanciare una **activity**
- Inviare dati a un **Broadcast Receiver**
- Avviare un **servizio**
- Aprire una **pagina web**

Contiene il riferimento **all'azione** e dei **dati** da elaborare

```
String myData = "content://contacts/people/8";

Intent myActivity2 = new Intent(Intent.ACTION_EDIT,
                                Uri.parse(myData));

startActivity(myActivity2);
```

Intent 2/3



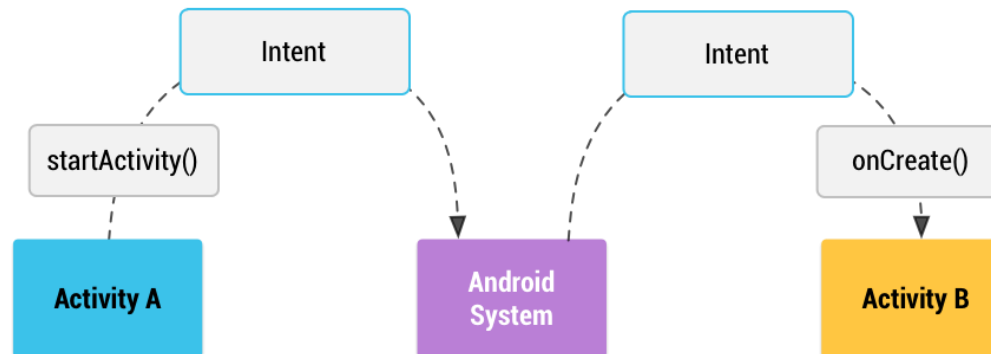
Possibili coppie azione/dati sono:

- *ACTION_DIAL tel:123*
 - visualizza la tastiera del telefono con il numero specificato inserito
- *ACTION_VIEW http://www.google.com*
 - visualizza la pagina di Google all'interno di un browser.
- *ACTION_EDIT content://contacts/people/2*
 - consente di digitare informazioni circa la persona il cui identificativo è 2
- *ACTION_VIEW content://contacts/people/2*
 - utilizzato per avviare una activity per la visualizzate informazioni circa la persona il cui identificativo è 2
- *ACTION_VIEW content://contacts/people/*
 - visualizza una lista di persone, attraverso cui l'utente può navigare. La selezione di un particolare contatto da visualizzare genererà un nuovo Intent (ACTION_VIEW content://contacts/people/N) per l'avvio di una nuova Activity

Un **Intent** può essere:

- **Esplicito** -> individua **univocamente il destinatario** attraverso un identificativo (ad esempio aprire la schermata delle impostazioni della propria app)
- **Implicito** -> dichiara un **compito generico** che deve essere eseguito senza identificare un destinatario univoco (ad esempio aprire una pagina web, non viene specificato il browser)

Se si vuole risolvere un Intent implicito nella propria applicazione, questo va dichiarato tramite un **Intent Filter**



Il **Bundle** è un contenitore che può essere usato per passare dati tra Activity e si può allegare ad un Intent come dati extra

- Può essere usato per salvare dati sulle activity prima che siano terminate

MainActivity.class

```
Intent intent = new Intent(this, SecondActivity.class);
Bundle bundle = new Bundle();
bundle.putString("key_1", "MainActivity greeted you with a HI");
bundle.putBoolean("key_2", true);
intent.putExtras(bundle);
startActivity(intent);
```



SecondActivity.class

```
Bundle bundle = getIntent().getExtras();
String title = bundle.getString("key_1");
boolean b = bundle.getBoolean("key_2");
```

Android Manifest



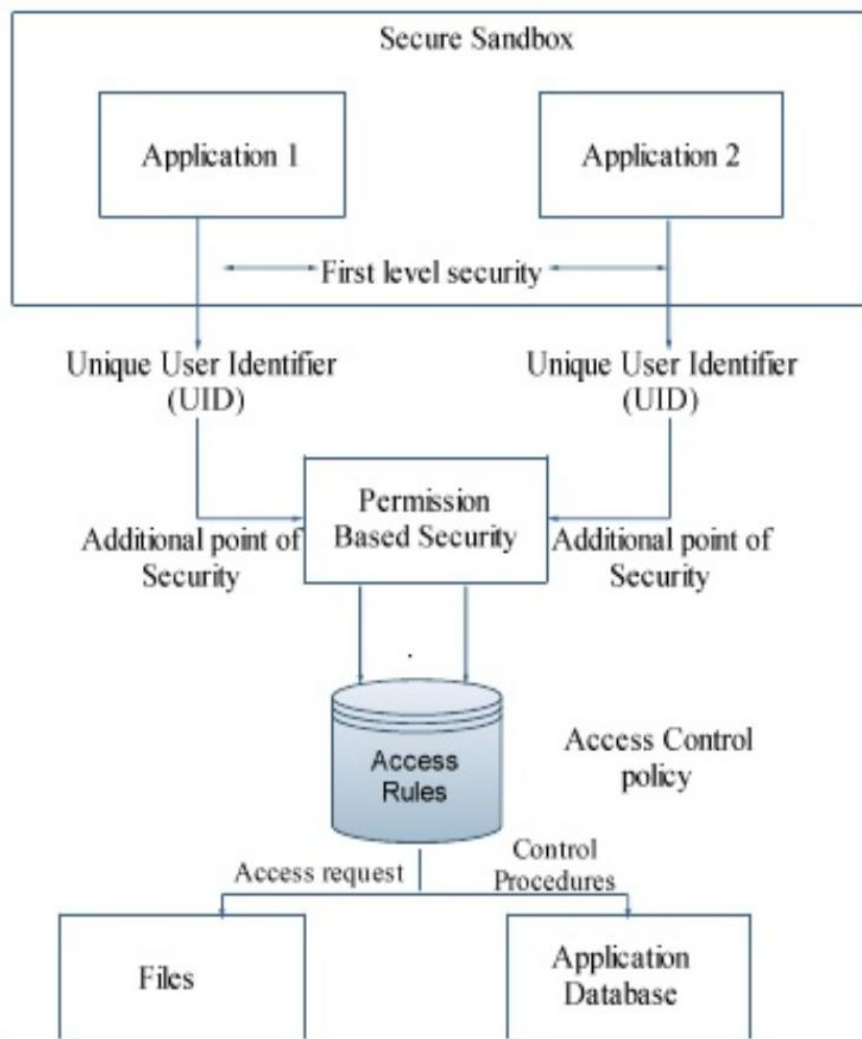
Il file *AndroidManifest.xml* è un documento XML che descrive le informazioni essenziali della applicazione

- Package e ID dell'applicazione
- Componenti app: Activity, Servizi, Receiver, Provider
- Intent filters
- Icone e label
- Permessi
- Compatibilità: Versione minima di Android richiesta

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:versionCode="1"
  android:versionName="1.0"
  package="com.example.myapplication"
  <uses-sdk android:minSdkVersion="15" android:targetSdkVersion="26" />

  <application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:label="@string/app_name"
    android:supportRtl="true"
    android:theme="@style/AppTheme">
    <activity android:name=".MainActivity">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>

    <activity
      android:name=".DisplayMessageActivity"
      android:parentActivityName=".MainActivity" />
    </activity>
  </application>
</manifest>
```

- Ogni applicazione è confinata all'interno della sua **sandbox** e non può interferire con il sistema esterno
- Lo sviluppatore deve esplicitamente richiedere i **permessi** per accedere alle **risorse di sistema** e ai dati di **altre applicazioni**
 - SMS, Accesso alla rete telefonica, Lettura/Scrittura file, Gestione energetica ...

I permessi possono essere

- **Predefiniti** -> accesso a **funzionalità standard** di Android (es. Invio SMS o accesso alla memoria interna)
- **Custom** -> propri di una applicazione e definiti all'interno del suo Manifest

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.package.Sample">
    <permission android:name="com.example.project.PERMISSION" ... />
    <permission-group ... />
    <application android:icon="@drawable/icon" android:label="@string/app_name">
    <activity android:name=".Sample"
        android:label="@string/app_name"
        android:permission="com.example.project.PERMISSION">
    </activity>
    </application>
    <uses-permission android:name="android.permission.RECEIVE_SMS" />
    <uses-permission android:name="com.example.project.PERMISSION" />
</manifest>
```

Nel Manifest di esempio si definisce un Permesso Custom (PERMISSION) che serve per accedere alla sua activity *.Sample*, inoltre chiede due permessi di cui uno predefinito (RECEIVE SMS) e uno custom (PERMISSION)

- I **permessi custom** vanno definiti nell'AndroidManifest.xml

```
<permission android:description="string resource"  
            android:icon="drawable resource"  
            android:label="string resource"  
            android:name="string"  
            android:permissionGroup="string"  
            android:protectionLevel=["normal" | "dangerous" |  
                                    "signature" | "signatureOrSystem"] />
```

- È possibile creare **gruppi di permessi** nel Manifest
 - Se chiedo il permesso all'utilizzo del gruppo è come se stessi chiedendo tutti i permessi contenuti nel gruppo

```
<permission-group android:description="string resource"  
                 android:icon="drawable resource"  
                 android:label="string resource"  
                 android:name="string" />
```

Android prevede diversi meccanismi per salvare dati utili alle applicazioni:

- **Preferenze** -> memorizza singole variabili all'interno di un array associativo
 - `getPreferences()` -> preferenze dell'activity
 - `getSharedPreferences()` -> preferenze dell'applicazione
 - `getDefaultSharedPreferences()` -> preferenze globali di Android
 - Le preferenze sono salvate in un area di storage privata in formato XML
- **Internal Storage**
 - Memoria interna del dispositivo condivisa, richiede permessi di lettura e scrittura espliciti da parte dell'utente
- **Scoped Internal Storage**
 - Cartella nella memoria interna del dispositivo, gestita da Android e visibile solo alla applicazione

Android prevede diversi meccanismi per salvare dati utili alle applicazioni:

- **SQLite** -> Android integra un database relazionale che consente la creazione di un database SQL, implementa la maggior parte dello standard implementa la maggior parte dello standard SQL-92
 - Ogni Applicazione ha un database privato -> per accedere all'esterno serve un content provider
 - Gran parte delle funzionalità sono disponibili: si possono creare database, definire tabelle, inserire tuple, modificare i dati ed eseguire query;
- **Network Connection** -> Salvare i dati sulla rete; java.net and android.net forniscono le classi per gestire l'accesso a dati via internet

- I Resource sono file esterni utilizzati internamente nel codice dell'applicazione
- Sono memorizzati all'interno del file .apk
- Android supporta diversi tipi di file, tra i quali **immagini** (PNG, JPEG ...) **documenti** XML e file **multimediali**

Directory	Resource type
res/anim/	File xml che descrivono delle animazioni
res/drawable/	Questa directory deve essere usata per le immagini necessarie al software. Sono supportati i formati PNG e JPEG
res/layout/	File XML per specificare il layout delle interface utente
res/values/	File XML che descrivono stringhe e altri parametri utilizzati dall'applicazione.
res/xml/	Questa directory ospita file XML di tipo qualsiasi, che il sistema interpreta e rende semplici da acquisire
res/raw/	Questa directory ospita file binari di tipo qualsiasi (ad esempio audio o filmati) che vengono copiati nell' .apk e l'applicazione poi carica

- **Concetti di Sistemi Operativi:** A. Silberschatz, P.B. Galvin, G. Gagne, Operating System Concepts, Wiley
- Android Studio
 - <https://developer.android.com/studio>
- Android Open Source Project:
 - <https://source.android.com/>
- Documentazione ufficiale Android SDK
 - <https://developer.android.com/>
- Anatomy & Physiology of an Android, Google I/O 2008